

Aula Detalhada - Algoritmos Procedurais, Assembly Básico E Operandos

Tema do dia: noção de algoritmo procedural, instruções assembly, registradores, imediatos, memória, leitura de pseudocódigo e tradução básica para operações de máquina **Aula na sequência:** 25 **Objetivo:** iniciar Arquitetura de Computadores entendendo como um algoritmo simples vira instruções, quais tipos de operandos aparecem em assembly e como registradores, constantes e memória participam da execução.

1. Onde Esta Aula Entra No Estudo?

Até agora, você estudou principalmente a base de Sistemas Digitais:

```
bits
portas lógicas
álgebra booleana
Karnaugh
MUX/decoder/encoder
latches e flip-flops
contadores
temporização
FSM
```

Agora começa outro bloco do edital:

```
Arquitetura de Computadores
```

A arquitetura fica entre dois mundos:

```
software escrito por pessoas
hardware digital que executa bits
```

O objetivo desta aula é criar a ponte:

```
algoritmo -> instruções assembly -> registradores/memória -> hardware
```

Você não precisa virar especialista em uma linguagem assembly específica.

O edital não fixa uma ISA como MIPS, RISC-V, ARM ou x86.

Então vamos usar uma notação genérica, parecida com arquiteturas RISC, só para entender os conceitos.

2. O Que É Arquitetura De Computadores?

Arquitetura de computadores estuda a interface entre o programa e a máquina.

Ela responde perguntas como:

```
quais instruções o processador entende?  
onde ficam os dados?  
como a ULA recebe operandos?  
como uma instrução acessa memória?  
como o processador decide desviar ou continuar?
```

Na prática, a arquitetura define um contrato.

De um lado:

```
programador / compilador
```

Do outro:

```
hardware
```

O contrato diz:

```
se você escrever esta instrução, o processador fará esta operação
```

Exemplo genérico:

```
ADD R1, R2, R3
```

Leitura:

```
R1 recebe R2 + R3
```

O hardware por baixo precisa:

```
ler R2  
ler R3  
somar na ULA  
guardar o resultado em R1
```

3. Camadas: Do Algoritmo Ao Hardware

Um programa pode ser visto em camadas.

```
algoritmo em alto nível  
-> linguagem de programação  
-> assembly  
-> código de máquina  
-> sinais digitais no processador
```

Exemplo:

```
x = a + b
```

Em assembly genérico:

```
ADD R1, R2, R3
```

Em código de máquina, isso vira bits.

Algo como:

```
opcode | registrador destino | registrador fonte 1 | registrador fonte 2
```

O formato exato depende da arquitetura, mas a ideia é:

```
assembly é uma representação legível das instruções de máquina
```

4. Algoritmo Procedural

Um algoritmo procedural é uma sequência de passos executados de forma organizada.

Ele costuma ter:

```
variáveis  
atribuições  
operações  
condições  
laços  
funções/procedimentos
```

Exemplo:

```
x = 3  
y = 4  
z = x + y
```

Esse algoritmo diz:

1. guarde 3 em `x`;

2. guarde 4 em `y`;
3. some `x` e `y`;
4. guarde o resultado em `z`.

O processador não entende diretamente "variável" como uma linguagem de alto nível.

Ele entende:

```
registradores  
memória  
instruções
```

5. Variáveis Em Alto Nível Versus Dados Na Máquina

Em uma linguagem de alto nível, você escreve:

```
idade = idade + 1
```

Na máquina, isso precisa virar algo mais concreto:

```
buscar o valor atual  
somar 1  
guardar o novo valor
```

Dependendo de onde `idade` está, pode envolver memória:

```
LOAD R1, [idade]  
ADDI R1, R1, 1  
STORE [idade], R1
```

Leitura:

```
carrega idade para R1  
soma 1 em R1  
salva R1 de volta em idade
```

O processador trabalha melhor com registradores.

A memória guarda muitos dados, mas é mais lenta.

Os registradores guardam poucos dados, mas são muito rápidos.

6. O Que É Assembly?

Assembly é uma linguagem de baixo nível que representa instruções do processador de forma legível.

Exemplo:

```
ADD R1, R2, R3
```

É mais legível do que bits puros, mas ainda está muito perto do hardware.

Uma instrução assembly costuma ter:

operação + operandos

Exemplo:

```
ADD R1, R2, R3
```

Parte	Significado
ADD	operação
R1	operando destino
R2	primeiro operando fonte
R3	segundo operando fonte

Leitura comum:

```
R1 = R2 + R3
```

7. Operação E Operandos

Toda instrução precisa dizer duas coisas:

o que fazer
com quais dados fazer

A operação é o que fazer.

Exemplos:

```
ADD -> somar
SUB -> subtrair
AND -> operação lógica AND
OR  -> operação lógica OR
XOR -> operação lógica XOR
LOAD -> carregar da memória
STORE -> salvar na memória
BEQ -> desviar se igual
```

Os operandos são os dados usados pela instrução.

Exemplos:

```
R1
R2
5
[1000]
[R3]
```

Os três tipos mais importantes nesta aula são:

```
registrador
imediato
memória
```

8. Operando Do Tipo Registrador

Registradores são pequenas posições de armazenamento dentro do processador.

Pense neles como variáveis muito rápidas do hardware.

Exemplos de nomes genéricos:

```
R0
R1
R2
R3
```

Uma instrução com registradores:

```
ADD R1, R2, R3
```

Leitura:

```
R1 = R2 + R3
```

Aqui:

```
R1 -> destino  
R2 -> fonte  
R3 -> fonte
```

8.1 Por que usar registradores?

Porque a ULA normalmente opera diretamente com valores vindos dos registradores.

Modelo simples:

```
banco de registradores -> ULA -> banco de registradores
```

Exemplo:

```
R2 = 7  
R3 = 5  
ADD R1, R2, R3
```

Depois da instrução:

```
R1 = 12
```

9. Operando Do Tipo Imediato

Um imediato é uma constante escrita dentro da própria instrução.

Exemplo:

```
ADDI R1, R2, 5
```

Leitura:

```
R1 = R2 + 5
```

O valor **5** não está em outro registrador.

Ele está embutido na instrução.

Outro exemplo:

```
ANDI R1, R2, 15
```

Leitura:

```
R1 = R2 AND 15
```

9.1 Quando o imediato aparece?

Imediatos aparecem quando o programa usa uma constante.

Alto nível:

```
x = x + 1
```

Assembly:

```
ADDI R1, R1, 1
```

Aqui, `1` é imediato.

9.2 Limitação do imediato

O imediato precisa caber no campo reservado dentro da instrução.

Exemplo conceitual:

```
campo imediato de 8 bits -> representa 256 combinações
```

Se a constante for grande demais, pode ser necessário carregá-la em partes ou buscá-la da memória.

Para a prova, a ideia mais importante é:

```
imediato = constante dentro da instrução
```

10. Operando Em Memória

Memória guarda instruções e dados.

Variáveis de programa normalmente ficam na memória, embora valores temporários fiquem em registradores.

Exemplo:

```
LOAD R1, [1000]
```

Leitura:

```
R1 recebe o valor guardado no endereço 1000
```

Outro exemplo:

```
STORE [1000], R1
```

Leitura:

o endereço 1000 recebe o valor de R1

10.1 Colchetes

Nesta aula, vamos usar colchetes para indicar acesso à memória.

[1000] -> conteúdo da memória no endereço 1000

[R2] -> conteúdo da memória no endereço guardado em R2

Sem colchetes:

R2 -> o valor do registrador R2

Com colchetes:

[R2] -> o valor da memória no endereço apontado por R2

Essa diferença é muito cobrável.

11. Load E Store

Muitas arquiteturas didáticas usam o modelo load/store.

Nesse modelo:

operações aritméticas usam registradores
acesso à memória é feito por LOAD e STORE

Exemplo:

```
LOAD R1, [A]  
LOAD R2, [B]  
ADD R3, R1, R2  
STORE [C], R3
```

Isso implementa:

$C = A + B$

Passo a passo:

Instrução	O que faz
LOAD R1, [A]	carrega A para R1
LOAD R2, [B]	carrega B para R2
ADD R3, R1, R2	soma R1 e R2 , guarda em R3
STORE [C], R3	salva R3 em C

Por que não fazer direto?

```
ADD [C], [A], [B]
```

Algumas arquiteturas permitem operações mais próximas disso.

Mas em arquiteturas RISC, o padrão é:

```
carrega da memória -> opera em registradores -> salva na memória
```

12. Instruções Aritméticas

Instruções aritméticas fazem contas.

Exemplos:

```
ADD R1, R2, R3
SUB R1, R2, R3
ADDI R1, R2, 4
```

Leituras:

Instrução	Leitura
ADD R1, R2, R3	$R1 = R2 + R3$
SUB R1, R2, R3	$R1 = R2 - R3$
ADDI R1, R2, 4	$R1 = R2 + 4$

Exemplo:

```
R2 = 10
R3 = 3
SUB R1, R2, R3
```

Resultado:

R1 = 7

13. Instruções Lógicas

Instruções lógicas operam bit a bit.

Exemplos:

```
AND R1, R2, R3
OR  R1, R2, R3
XOR R1, R2, R3
NOT R1, R2
```

Exemplo:

```
R2 = 1100
R3 = 1010
AND R1, R2, R3
```

Resultado:

R1 = 1000

Porque:

```
  1100
& 1010
-----
  1000
```

Outro exemplo:

```
R2 = 1100
R3 = 1010
OR  R1, R2, R3
```

Resultado:

R1 = 1110

Essas instruções conversam diretamente com as primeiras aulas de portas lógicas.

14. Instruções De Comparação E Desvio

Um programa precisa tomar decisões.

Em alto nível:

```
if (a == b) {  
    x = 1  
} else {  
    x = 0  
}
```

Em assembly, isso vira comparação e desvio.

Exemplo genérico:

```
BEQ R1, R2, igual  
ADDI R3, R0, 0  
J fim  
igual:  
ADDI R3, R0, 1  
fim:
```

Leitura:

```
se R1 == R2, desvie para igual  
senão, continue na próxima instrução
```

Onde:

```
BEQ -> branch if equal  
J    -> jump
```

Não se preocupe ainda em decorar sintaxe exata.

O importante é entender:

```
condição em alto nível vira teste + desvio
```

15. Labels

Um label é um nome dado a um ponto do código.

Exemplo:

```
igual:
ADDI R3, R0, 1
```

O label `igual` marca o endereço daquela instrução.

Quando uma instrução diz:

```
BEQ R1, R2, igual
```

ela quer dizer:

```
se R1 for igual a R2, pule para o ponto chamado igual
```

Labels ajudam a representar:

```
if
else
loops
funções
```

16. Laços Em Assembly

Um laço repete um trecho de código.

Alto nível:

```
while (i < n) {
    i = i + 1
}
```

Em assembly genérico, a estrutura vira:

```
loop:
    comparar i com n
    se condição falhar, sair
    corpo do laço
    voltar para loop
fim:
```

Exemplo simplificado usando `BEQ`:

```
loop:
BEQ R1, R2, fim
ADDI R1, R1, 1
J loop
fim:
```

Leitura:

enquanto R1 não for igual a R2, some 1 em R1
quando R1 == R2, saia do laço

Isso implementa a ideia:

```
while (R1 != R2) {
    R1 = R1 + 1
}
```

17. Chamada De Função Em Nível Conceitual

O edital menciona chamadas de funções.

Vamos deixar aqui uma primeira visão, sem entrar ainda em pilha e convenções completas.

Uma chamada de função precisa:

pular para o código da função
guardar para onde deve voltar
passar argumentos
receber resultado

Em assembly genérico:

```
CALL soma
```

Significa:

vá para a função soma e guarde o endereço de retorno

Depois, no fim da função:

```
RET
```

Significa:

volte para a instrução depois da chamada

Conceitos importantes:

Conceito	Ideia
endereço de retorno	onde continuar depois da função
argumentos	valores enviados para a função
valor de retorno	resultado produzido pela função
pilha	região de memória usada para salvar contexto

Vamos aprofundar isso nas próximas aulas de arquitetura.

18. Exemplo Completo 1 - Atribuição E Soma

Alto nível:

```
c = a + b
```

Suponha:

```
a, b e c estão na memória
```

Assembly genérico:

```
LOAD R1, [a]
LOAD R2, [b]
ADD R3, R1, R2
STORE [c], R3
```

Passo a passo:

Passo	Instrução	Efeito
1	LOAD R1, [a]	R1 = a
2	LOAD R2, [b]	R2 = b
3	ADD R3, R1, R2	R3 = R1 + R2
4	STORE [c], R3	c = R3

Se:

```
a = 7
b = 5
```

então:

```
c = 12
```

19. Exemplo Completo 2 - Constante Imediata

Alto nível:

```
x = x + 10
```

Se `x` está na memória:

```
LOAD R1, [x]
ADDI R1, R1, 10
STORE [x], R1
```

O operando `10` é imediato.

Ele não precisou ser carregado da memória.

Leitura:

```
R1 recebe x
R1 recebe R1 + 10
x recebe R1
```

20. Exemplo Completo 3 - If/Else

Alto nível:

```
if (a == b) {
    x = 1
} else {
    x = 0
}
```

Suponha que:

```
a está em R1
b está em R2
x será guardado em R3
R0 vale 0
```

Assembly genérico:

```
BEQ R1, R2, igual
ADDI R3, R0, 0
J fim
igual:
ADDI R3, R0, 1
fim:
```

Rastreio:

Situação	Caminho
<code>R1 == R2</code>	desvia para <code>igual</code> , faz <code>R3 = 1</code>
<code>R1 != R2</code>	não desvia, faz <code>R3 = 0</code> , pula para <code>fim</code>

Esse exemplo mostra como o `if/else` vira:

```
desvio condicional + desvio incondicional
```

21. Exemplo Completo 4 - Laço

Alto nível:

```
while (i != n) {
    i = i + 1
}
```

Suponha:

```
i está em R1
n está em R2
R0 vale 0
```

Assembly:

```
loop:
BEQ R1, R2, fim
ADDI R1, R1, 1
J loop
fim:
```

Exemplo com:

```
R1 = 2
R2 = 5
```

Rastreio:

Iteração	R1 antes	R2	Comparação	Ação
1	2	5	diferente	R1=3
2	3	5	diferente	R1=4
3	4	5	diferente	R1=5
4	5	5	igual	sai

Valor final:

```
R1 = 5
```

22. Relação Com Sistemas Digitais

Assembly parece software, mas por baixo ele conversa com hardware digital.

Exemplo:

```
ADD R1, R2, R3
```

Por baixo:

```
banco de registradores lê R2 e R3
ULA soma
banco de registradores escreve R1
```

Isso usa assuntos que você já viu:

Aula anterior	Relação com arquitetura
MUX	seleciona operandos e caminhos no datapath
Registradores	guardam operandos e resultados
ULA	usa portas e blocos aritméticos
Clock	sincroniza escrita em registradores
FSM	pode implementar a unidade de controle

Ou seja, arquitetura não é um assunto completamente novo.

É uma camada acima dos circuitos digitais.

23. Como Questões Costumam Cobrar

23.1 Identificar operandos

Pergunta:

Em `ADDI R1, R2, 5`, quais operandos são registradores e qual é imediato?

Resposta:

R1 e R2 são registradores
5 é imediato

23.2 Ler uma instrução

Pergunta:

O que faz `SUB R4, R1, R2`?

Resposta:

$R4 = R1 - R2$

23.3 Diferenciar registrador e memória

Pergunta:

Qual a diferença entre R1 e [R1]?

Resposta:

R1 é o valor do registrador
[R1] é o valor na memória no endereço guardado em R1

23.4 Traduzir pseudocódigo

Pergunta:

$x = y + 3$

Possível resposta:

```
LOAD R1, [y]
ADDI R1, R1, 3
STORE [x], R1
```

23.5 Entender desvio

Pergunta:

BEQ R1, R2, alvo

Resposta:

se $R1 == R2$, o fluxo pula para alvo

24. Erros Comuns

24.1 Confundir imediato com endereço de memória

Em:

```
ADDI R1, R2, 5
```

o 5 é uma constante.

Não significa memória no endereço 5.

Para indicar memória, usamos:

[5]

24.2 Confundir registrador com conteúdo da memória

R1

é o valor no registrador.

[R1]

é o valor na memória no endereço apontado por R1.

24.3 Achar que toda variável fica sempre em memória

Variáveis podem estar temporariamente em registradores.

O compilador tenta usar registradores porque são mais rápidos.

24.4 Ler destino e fonte na ordem errada

Nesta aula usamos:

OP destino, fonte1, fonte2

Exemplo:

SUB R1, R2, R3

Leitura:

R1 = R2 - R3

Não é:

R3 = R1 - R2

Cuidado: algumas arquiteturas usam sintaxe diferente.

Em prova, leia o padrão dado no enunciado.

24.5 Esquecer o STORE

Se uma variável está na memória, modificar só o registrador não altera automaticamente a memória.

Exemplo:

LOAD R1, [x]
ADDI R1, R1, 1

Depois disso, **R1** foi alterado.

Mas para salvar em **x**, precisa:

```
STORE [x], R1
```

25. Exercícios

Exercício 1

O que é uma instrução assembly?

Exercício 2

Na instrução abaixo, identifique operação, destino e fontes:

```
ADD R1, R2, R3
```

Exercício 3

Na instrução abaixo, qual operando é imediato?

```
ADDI R4, R4, 8
```

Exercício 4

Explique a diferença entre:

```
R2  
[R2]
```

Exercício 5

O que faz a instrução?

```
LOAD R1, [x]
```

Exercício 6

O que faz a instrução?

```
STORE [y], R3
```

Exercício 7

Traduza para leitura matemática:

```
SUB R5, R1, R2
```

Exercício 8

Considere:

```
R1 = 6  
R2 = 10  
ADD R3, R1, R2  
ADDI R3, R3, 4
```

Qual é o valor final de **R3**?

Exercício 9

Traduza para assembly genérico:

```
c = a + b
```

Suponha que **a**, **b** e **c** estão na memória.

Exercício 10

O que faz:

```
BEQ R1, R2, igual
```

Exercício 11

Rastreie:

```
R1 = 1
R2 = 3
loop:
BEQ R1, R2, fim
ADDI R1, R1, 1
J loop
fim:
```

Qual é o valor final de **R1** ?

Exercício 12

Por que uma operação aritmética normalmente usa registradores antes de salvar o resultado na memória?

26. Gabarito Comentado

Gabarito 1

É uma representação legível de uma instrução de máquina.

Ela indica uma operação e seus operandos.

Gabarito 2

```
operação = ADD
destino = R1
fonte 1 = R2
fonte 2 = R3
```

Leitura:

```
R1 = R2 + R3
```

Gabarito 3

O imediato é:

```
8
```

Gabarito 4

R2 -> valor armazenado no registrador R2
[R2] -> valor na memória no endereço contido em R2

Gabarito 5

Carrega o conteúdo da memória identificado por `x` para `R1`.

```
R1 = memória[x]
```

Gabarito 6

Salva o valor de `R3` na posição de memória identificada por `y`.

```
memória[y] = R3
```

Gabarito 7

```
R5 = R1 - R2
```

Gabarito 8

Primeiro:

```
R3 = 6 + 10 = 16
```

Depois:

```
R3 = 16 + 4 = 20
```

Resposta:

```
R3 = 20
```

Gabarito 9

Uma tradução possível:

```
LOAD R1, [a]
LOAD R2, [b]
ADD R3, R1, R2
STORE [c], R3
```

Gabarito 10

Se `R1 == R2`, o fluxo de execução desvia para o label `igual`.

Se não forem iguais, a execução continua na próxima instrução.

Gabarito 11

Rastreio:

```
R1 = 1, R2 = 3 -> diferente -> R1 = 2
R1 = 2, R2 = 3 -> diferente -> R1 = 3
R1 = 3, R2 = 3 -> igual -> sai
```

Valor final:

```
R1 = 3
```

Gabarito 12

Porque registradores são rápidos e ficam próximos da ULA.

Em arquiteturas load/store, a ULA opera principalmente sobre registradores.

Depois de calcular, o resultado pode ser salvo na memória com `STORE`.

27. O Que Memorizar

Memorize:

```
assembly = forma legível das instruções de máquina
instrução = operação + operandos
registrador = armazenamento rápido dentro do processador
imediato = constante dentro da instrução
memória = armazenamento acessado por endereço
```

Diferença importante:

R1 -> valor do registrador
[R1] -> valor da memória no endereço guardado em R1

Modelo load/store:

LOAD -> memória para registrador
STORE -> registrador para memória
ADD/SUB/AND/OR -> operação em registradores

Condição:

if/else -> comparação + desvio
loop -> label + condição + salto de volta

28. Plano De Estudo Para Esta Aula

Sugestão:

Etapa	Tempo	O que fazer
Conceitos	20 min	Ler seções 1 a 10
Operandos	20 min	Praticar registrador, imediato e memória
Exemplos	30 min	Refazer os exemplos 18 a 21
Exercícios	35 min	Resolver os 12 exercícios
Revisão	10 min	Fazer uma lista curta com <code>LOAD</code> , <code>STORE</code> , <code>ADD</code> , <code>BEQ</code>

Se tiver pouco tempo, foque em:

registrador vs imediato vs memória
LOAD e STORE
ADD/ADDI
BEQ e J

29. Conexão Com A Próxima Aula

A próxima aula deve aprofundar:

- formatos e tipos de instrução
- opcode
- campos de registrador
- campo imediato
- instruções aritméticas/lógicas
- instruções condicionais
- desvios e chamadas de função

Ou seja, nesta aula você aprendeu a ler o significado das instruções.

Na próxima, você vai ver como essas instruções são organizadas em campos e como isso conversa com o hardware.